

13_ROS2 파일 시스템

2101080 정진용

1.

실습과제 1

- 빌드 시스템과 빌드 툴의 차이를 설명하라.
- 패키지 생성 명령어를 실행하는 위치는 어디이고 그곳으로 이동하는 명령어를 쓰시오.
- 패키지 생성 명령어의 사용법을 설명하라.
- 패키지 빌드 명령어를 실행하는 위치는 어디이고 그곳으로 이동하는 명령어를 쓰시오.
- 패키지 빌드 명령어의 사용법을 설명하라.

1. 빌드 시스템과 빌드 툴의 차이

(ANSWER)

빌드 시스템과 빌드 툴의 가장 큰 차이는 빌드 대상의 범위입니다.

빌드 시스템은 단일 패키지를 대상으로 동작하며, 해당 패키지의 의존성을 해결하고 빌드하여 실행 파일을 생성하는 역할을 합니다. ROS 2에서는 C++ 패키지를 위한

ament_cmake와 Python 패키지를 위한 ament_python이 빌드 시스템으로 사용됩니다. 반면,

빌드 툴은 시스템 전체를 대상으로 하며, 수많은 패키지들의 의존성 관계를 해석하고 올바른 순서대로 각 패키지의 빌드 시스템을 호출하는 역할을 합니다. 즉,

colcon과 같은 빌드 툴은 서로 다른 빌드 시스템을 사용하는 여러 패키지들을 통합하여 관리하고 빌드합니다.

2. 패키지 생성 명령어 실행 위치 및 이동 명령어

(ANSWER)

패키지 생성 명령어인 `ros2 pkg create`는 반드시 사용자 작업 폴더 내의 `src` 폴더에서 실행해야 합니다. 해당 위치는 `~/ros2_ws/src`입니다. 이동 명령어는 `cd ~/ros2_ws/src`입니다.

3. 패키지 생성 명령어 사용법 (ANSWER)

```
ros2 pkg create <pkgname> --build-type <buildtype> --dependencies <deppkg1> ...
```

<pkgname>: 생성하려는 패키지의 이름을 지정합니다.

<buildtype>: 패키지의 빌드 타입을 명시하며, C++는 ament_cmake를, Python은 ament_python을 사용합니다.

<deppkgx>: 해당 패키지를 빌드하는 데 필요한 의존성 라이브러리(패키지)들을 의미합니다.

4. 패키지 빌드 명령어 실행 위치 및 이동 명령어 (ANSWER)

패키지 빌드 명령어인 colcon build는 반드시 워크스페이스의 최상위 폴더(~/.ros2_ws)로 이동하여 실행해야 합니다. 이동 명령어는 cd ~/.ros2_ws입니다.

5. 패키지 빌드 명령어 사용법 (ANSWER)

특정 또는 전체 패키지를 빌드할 때는 colcon 빌드 툴을 사용합니다.

colcon build 명령어 실행 후, 워크스페이스 아래에 build, install, log 폴더가 생성됩니다.

```
colcon build --symlink-install --packages-select <pkgname>
```

주요 옵션:

--packages-select <pkgname>: 전체 패키지가 아닌, 명시된 특정 패키지만을 선택하여 빌드할 때 사용합니다.

--symlink-install: 설치 시 실제 파일을 복사하는 대신 심볼릭 링크를 생성하여, 소스 코드를 수정한 후 다시 빌드하지 않고도 변경 사항이 즉시 반영되게 할 수 있습니다.

2.

실습과제 2

- 사용자 홈디렉터리 아래에 작업폴더 ros2_ws를 생성하고 강의노트 의 패키지 생성 및 빌드 명령어를 실습하고 결과를 제출하시오.
- 자동으로 생성되는 파일과 디렉터리를 출력하고 각각 설명하시오.

(ANSWER)

```
linux@JINYOUNG:~$ mkdir ros2_wss
linux@JINYOUNG:~/ros2_wss$ mkdir src
linux@JINYOUNG:~/ros2_wss$ cd src/
linux@JINYOUNG:~/ros2_wss/src$ ros2 pkg create first_pkg --build-type ament_cmake
--dependencies rclcpp std_msgs
going to create a new package
package name: first_pkg
destination directory: /home/linux/ros2_wss/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['linux <linux@todo.todo>']
licenses: ['TODO: License declaration']
build type: ament_cmake
dependencies: ['rclcpp', 'std_msgs']
creating folder ./first_pkg
creating ./first_pkg/package.xml
creating source and include folder
creating folder ./first_pkg/src
creating folder ./first_pkg/include/first_pkg
creating ./first_pkg/CMakeLists.txt
linux@JINYOUNG:~/ros2_wss/src$ ls
linux@JINYOUNG:~/ros2_wss/src$ cd first_pkg/
linux@JINYOUNG:~/ros2_wss/src/first_pkg$ tree -L 1
.
├── CMakeLists.txt
├── include
├── package.xml
└── src
```

1. CMakeLists.txt: C++ 프로젝트를 빌드하는 방법을 정의하는 빌드 설정 파일입니다. ROS 2의
2. package.xml: 패키지의 모든 정보를 담고 있는 설정 파일입니다. 패키지 이름, 버전, 작성자, 라이선스, 그리고 rclcpp와 같은 의존성 패키지 정보가 XML 형식으로 기록되어 있습니다.
3. include/: C++ **헤더 파일(.hpp, .h)**을 저장하는 폴더입니다.
4. src/: C++ **소스 코드 파일(.cpp)**을 저장하는 폴더입니다.

```
linux@JINYOUNG:~/ros2_wss$ colcon build --symlink-install --packages-select first_pkg
Starting >>> first_pkg
[Processing: first_pkg]
Finished <<< first_pkg [33.1s]

Summary: 1 package finished [34.0s]
linux@JINYOUNG:~/ros2_wss$ ls
build install log src
linux@JINYOUNG:~/ros2_wss$ tree -L 1
.
├── build
├── install
├── log
└── src

4 directories, 0 files
```

1. build: 빌드 과정에서 생성되는 중간 파일 및 설정 파일들이 저장되는 폴더입니다.
2. install: 빌드의 최종 결과물이 설치되는 폴더입니다. 컴파일된 실행 파일, 라이브러리, 헤더 파일 등이 이곳에 저장되어 실제 사용 준비를 마칩니다.
3. log: 빌드 과정에서 발생하는 모든 기록(로그) 파일이 저장되는 폴더입니다. 빌드 중 에러가 발생했을 때 원인을 찾기 위해 사용됩니다.
4. src: 사용자가 작성하거나 다운로드한 패키지의 소스 코드가 위치하는 폴더입니다.